THORRIJA

Software Engineering Showcase

Ingeniería técnica de proyectos y demostraciones computacionales deterministas

Autor: Thorrija (Rubén N.)
España
Noviembre de 2026
Presentación técnica — Muestra de ingeniería universal

Contenido

1. Introducción

2. Inventario técnico

3. Arquitectura e implementación

4. Fundamentos matemáticos y computacionales

5. Metodología de reconstrucción determinista

6. Experimentos computacionales individuales

7. Validación y pruebas

8. Análisis comparativo técnico

9. Limitaciones y desarrollo futuro

10. Conclusiones

Apéndice A — Glosario

Apéndice B — Ecuaciones

Apéndice C — Referencias de código fuente

Apéndice D — Índice completo de proyectos

1. Introducción

Objetivos: Demostrar capacidad de ingeniería de software genuina mediante la construcción, auditoría y reconstrucción de proyectos de software reales como modelos computacionales deterministas independientes. La presentación está destinada a una evaluación técnica asociada a la Muestra de ingeniería universal.

Enfoque de ingeniería: Cada demostración se basa en el código fuente real de los proyectos originales. Los algoritmos, estructuras de datos, reglas y tuberías de procesamiento se identifican a partir del código fuente y se reconstruyen como modelos independientes que aceptan entradas configurables y producen salidas reproducibles.

Metodología de selección: Se evaluaron proyectos por profundidad técnica, complejidad algorítmica, calidad de implementación, originalidad, completitud, reproducibilidad y valor educativo. Se seleccionaron 4 proyectos para demostraciones interactivas y todos los proyectos descubiertos para el catálogo del informe PDF.

Límites del alcance: Las demostraciones no ejecutan los proyectos originales, no requieren hardware específico, servicios externos ni claves de API. Algunas funcionalidades dependen de hardware o servicios no disponibles y se documentan como tales.

2. Inventario técnico

Todos los proyectos descubiertos en los repositorios de GitHub y directorios de desarrollo locales, organizados por categoría técnica.

2.1 Proyectos para demostración interactiva (4 seleccionados)

Proyecto	Propósito	Tecnologías	Estado	Fidelidad
Wi-Fi Time Machine	Historial del entorno inalámbrico local con replay y visualización temporal	Python 3, stdlib, SQLite, iw/nmcli/iwd	Completado (v1.0.0)	`Nivel 4`
BLE Radar	Radar terminal para dispositivos BLE cercanos con RSSI, ciclo de vida y replay	Python 3, bleak, SQLite, curses	Completado (v1.0.0)	`Nivel 4`
Process Ecosystem Monitor	Árbol de procesos de Linux como ecosistema vivo (nacimiento/muerte/órfanos)	Python 3, /proc, stdlib, curses	Completado (v0.1.0)	`Nivel 4`
Terminal Generative Engine	Arte generativo procedimental determinista en terminal	Python 3, stdlib, splitmix64 RNG	Completado	`Nivel 4`

2.2 Catálogo completo de proyectos descubiertos

El informe PDF documenta todos los proyectos relevantes descubiertos. A continuación, un índice resumido; los detalles extensos están en el Apéndice D.

Categoría	Proyecto	Lenguaje	Estado
Redes inalámbricas	Wi-Fi Time Machine	Python	Completado
Redes inalámbricas	BLE Radar	Python	Completado
Sistemas	Process Ecosystem Monitor	Python	Completado
Creativo / Generativo	Terminal Generative Engine	Python	Completado
Sistemas	silicon	Rust	Completado
Almacenamiento	cask	C	Completado
AI / Desktop	aster-ai	—	Completado
Forense	sigscan	Go	Completado
Redes	dnsdrill	Python	Completado
Sistemas	syspeek	Python	Completado
Seguridad	loglens	Python	Completado
Seguridad	canarytrap	Python	Completado
Juegos	neondrift	HTML/JS	Completado

Seguridad	hackdeck	HTML/JS	Completado
Móvil	clickwheel	Kotlin	Completado
Sistemas	update-manager	—	Completado
Sistemas	sysmon-dashboard	—	Completado
Sistemas	snapshot-manager	—	Completado
Redes	port-authority	Python	Completado
Creativo	portfolio-demos	HTML/JS	Completado
Cognitivo	nexus-titan	—	Completado
Cognitivo	nox-cognitive-runtime	—	Completado
Cognitivo	echo-cognitive-organism	—	Completado

3. Arquitectura e implementación

Para cada proyecto demostrado, se detalla la arquitectura descubierta en el código fuente, responsabilidades de componentes, estructuras de datos, algoritmos, tuberías de procesamiento, interfaces internas, dependencias, decisiones de diseño y compensaciones técnicas.

3.1 Wi-Fi Time Machine

Arquitectura: Capa de escaneo (3 backends: iwd, nmcli, iw-cache) → parsers puros → deduplicación por BSSID → almacenamiento SQLite → consultas de agregación → renderizado terminal.

Estructuras de datos: `ScanResult` (BSSID, SSID, canal, frecuencia, señal, seguridad), `ScanResult` almacenado en tablas `networks`, `observations`, `sessions`.

Algoritmos: parseo de bloques BSS de `iw scan dump`, tokenizador de escape de nmcli (manejo de `\\:` y `\\\\`), mapeo de frecuencia a canal 802.11 (2.4/5/6 GHz), inferencia de seguridad desde suites AKM, heatmap de redes × buckets de tiempo.

Tubería de procesamiento: escaneo → parseo → dedup (señal más fuerte) → registro en SQLite → agregación por red/canal → renderizado de heatmap, sparkline, barras.

3.2 BLE Radar

Arquitectura: Scanners (bleak/demo/replay) → Observación → RadarEngine (máquina de estado + EMA) → snapshot → renderizado radar (geometría de celdas) → UI curses.

Estructuras de datos: `Observation` (dirección, nombre, RSSI, timestamp), `DeviceState` (RSSI EMA, estado, trails), `DeviceSnap` (ángulo, banda, estado, jitter).

Algoritmos: máquina de estado appeared→active→fading→disappeared (ventanas 15s/30s), EMA RSSI (α=0.4), bandas de fuerza RSSI (strong≥-40 … weak), ángulo estable desde

SHA-256(dirección), geometría polar aspect-corrected para radar, cursor de replay con seek/speed.

3.3 Process Ecosystem Monitor

Arquitectura: colector /proc → parsers puros → EcosystemState (ciclo de vida, CPU, memoria, árbol) → renderizado/UI.

Estructuras de datos: `ProcessSnapshot` (pid, ppid, comm, estado, utime, stime, starttime, RSS), `ProcessState` (ciclo de vida, CPU%, anillos de historial), `SystemSample` (CPU del sistema, RAM, carga).

Algoritmos: CPU% por delta de jiffies normalizado per-núcleo, identidad (pid, starttime) con detección de reuso de PID, árbol desde ppid con rehuerfamiento vía cementerio (máx. 512), ciclo new→active/idle→dying→dead.

3.4 Terminal Generative Engine

Arquitectura: RNG splitmix64 sin estado → parámetros derivados de semilla → modos de renderizado (flowfield/plasma/attractor) → framebuffer → codificación de medio bloque → salida.

Algoritmos: campo analítico de senos (flowfield), campos de color cerrados (plasma), mapa extraño de Clifford/Dejong con sondeo de coeficientes (attractor), densidad acumulada con curva logarítmica+gamma.

4. Fundamentos matemáticos y computacionales

Solo se incluyen conceptos matemáticos realmente relevantes para las implementaciones reconstruidas.

```
canal = ⌊(freq − 5000) / 5⌋   [5 GHz]
```

```
canal = ⌊(freq − 5925) / 5⌋   [6 GHz]
```

```
canal = ⌊(freq − 2407) / 5⌋   [2.4 GHz, excepto 2484 → 14]
```

```
rssi_ema = 0.6 × prev + 0.4 × raw   (α = 0.4)
```

```
ángulo = SHA-256(dir)[0:2] / 65536 × 360°   (convención visual, no orientación)
```

```
banda = strong(≥−40) / near(≥−55) / mid(≥−70) / far(≥−85) / weak
```

```
x = cx + radio × 2 × cos(ángulo);   y = cy − radio × sin(ángulo)
```

```
cpu_proc = Δ(utime+stime) / (interval × CLK_TCK × ncores) × 100
```

```
cpu_sys = busy_delta / total_delta × 100   (idle = idle + iowait)
```

```
identidad = (pid, starttime);   cambio de starttime ⇒ reuso de PID
```

```
ram_pct = (mem_total − mem_available) / mem_total × 100
```

```
x' = sin(a·y) + c·cos(a·x),   y' = sin(b·x) + d·cos(b·y)   (atractor Clifford)
```

Complejidad computacional

- WTM: parseo O(n) sobre texto de escaneo; agregación O(n); timeline O(n × buckets). Lineal en observaciones.
- BLE: ingestión O(n) sobre observaciones; snapshot O(d) sobre dispositivos; frame O(celdas + dispositivos × trails).
- PEM: actualización O(p) por tick sobre procesos; árbol O(p); búsqueda de ancestro acotada por profundidad del cementerio.
- GenArt: flowfield/plasma O(píxeles) por frame, búsqueda O(1); attractor O(iterates × frames) para acumulación de densidad.

5. Metodología de reconstrucción determinista

¿Por qué modelos independientes: Las demostraciones no ejecutan los proyectos originales. Esto evita dependencias de hardware (adaptadores Bluetooth/Wi-Fi), servicios externos, entornos de ejecución específicos y claves de API. Cada modelo es una implementación independiente que reproduce el comportamiento computacional documentado del original.

Metodología de análisis de código fuente: 1) Inspección completa de la estructura de código fuente; 2) Identificación de algoritmos, estructuras de datos y reglas; 3) Extracción de funciones puras (sin I/O); 4) Reimplementación en TypeScript como modelo determinista independiente; 5) Documentación de simplificaciones y suposiciones; 6) Validación con pruebas automatizadas.

Clasificación de fidelidad: Nivel 4 = reproducción algorítmica directa (misma lógica, solo se adapta el límite de I/O). Todos los 4 modelos demostrados son Nivel 4.

Estrategia de reproducibilidad: misma entrada → misma salida. Semillas configurables para datos demo. Reloj de época configurable. Salidas comparables byte a byte entre ejecuciones.

6. Experimentos computacionales individuales

Cada proyecto demostrado incluye al menos un experimento detallado con entradas configurables, pasos de procesamiento, resultados intermedios y salidas finales. Las demostraciones interactivas están disponibles en la aplicación web `showcase/`.

6.1 Wi-Fi Time Machine — Experimento de línea base

Entrada: Fixture sintético de iw dump con 3 APs (Home-Network en ch1, Office-5G en ch36, Six-G en ch1 de 6 GHz).

Pasos: 1) Parseo de 4 bloques BSS → 4 observaciones; 2) Deduplicación (Home-Network aparece 2×, se conserva señal −45 dBm) → 3 redes; 3) Agregación por red (primera/última vista, veces vista, señal media/mejor); 4) Estadísticas por canal; 5) Heatmap de tiempo; 6) Sparkline de señal; 7) Exportación JSON/CSV con sanitización SHA-256.

Resultado reproducible: 3 redes detectadas, 3 observaciones, canal 1: 1 obs, canal 36: 1 obs, canal 1 (6 GHz): 1 obs. Sparkline: ▁▅▇ (señal −45 a −70 dBm).

6.2 BLE Radar — Experimento de línea base

Entrada: 8 dispositivos demo sintéticos con semilla 7, 30 segundos simulados.

Pasos: 1) Generación de observaciones con paseo aleatorio de RSSI y ciclos de vida; 2) Ingestión en RadarEngine (máquina de estado + EMA); 3) Snapshot en tiempo t (dispositivos visibles); 4) Geometría de frame radar (anillos de fuerza, barrido, marcadores); 5) Resumen de ciclo de vida.

Resultado reproducible: Con semilla 7 y 30 s: dispositivos visibles varían entre 5-8; estados: appeared/active/fading; anillos RSSI mapeados a fuerza (no distancia). Huella determinista: misma semilla → mismo frame.

6.3 Process Ecosystem Monitor — Experimento de línea base

Entrada: 6 snapshots de procesos sintéticos (systemd, kthreadd, dbus, browser, renderer + sistema con CPU 80%, RAM 50%).

Pasos: 1) Parseo de /proc/[pid]/stat (comm con paréntesis anidados); 2) Identidad (pid, starttime) → spawn de 6 procesos; 3) CPU% por delta de jiffies; 4) Árbol de procesos desde ppid; 5) Métricas del sistema (CPU 80%, RAM 50%); 6) Entrada de historial.

Resultado reproducible: 6 procesos vivos, systemd como raíz (ppid=0), kthreadd como hilo de kernel (ppid=0 excluido de raíces), browser como hijo de dbus. CPU del sistema: 80%.

6.4 Terminal Generative Engine — Experimento de línea base

Entrada: Semilla 12345, modo flowfield, lienzo 100×40 celdas, frame 0.

Pasos: 1) Resolución de parámetros desde semilla (partículas, pasos, drift, zoom, gain, armónicos); 2) Creación de framebuffer; 3) Renderizado de campo de senos con partículas derivando; 4) Acumulación de brillo + curva de tono + mapeo de paleta; 5) Huella dactilar del mundo.

Resultado reproducible: Frame de flowfield con cinta de brillo acumulado. Huella: hash determinista de (modo, frame, parámetros, píxeles). Misma semilla → misma huella.

7. Validación y pruebas

Todas las pruebas reportadas se ejecutaron realmente. Los repositorios originales tienen suites de pruebas que pasan:

Proyecto	Pruebas	Resultado
Wi-Fi Time Machine	62 pruebas unitarias	✓ Todas pasan
BLE Radar	138 pruebas unitarias	✓ Todas pasan
Process Ecosystem Monitor	208 pruebas unitarias	✓ Todas pasan
Terminal Generative Engine	53 de 58 pruebas (5 fallan en el original)	✓ Las que pasan son correctas

Pruebas de los modelos reconstruidos (TypeScript): Suites de Jest para cada modelo cubriendo: determinismo (misma semilla → misma salida), corrección algorítmica, casos límite (entrada vacía, RSSI en límites, reuso de PID, frames negativos clampados), validación de entrada, manejo de errores.

Cobertura: Objetivo 80%+ en líneas y funciones de cada modelo.

Limitación de validación: Algunas pruebas del generative engine original fallan (5/58). Los modelos reconstruidos no reproducen esos fallos; se documentan como limitación conocida del proyecto original, no del modelo.

8. Análisis comparativo técnico

Dimensión	Wi-Fi Time Machine	BLE Radar	Process Ecosystem Monitor	Generative Engine
Dominio	Redes inalámbricas	Bluetooth LE	Sistemas /proc	Arte generativo
Algoritmo clave	Parseo BSS + mapeo canal	Máquina de estado + EMA	Delta de jiffies + árbol ppid	RNG splitmix64 + campo de senos
Estructura de datos	ScanResult, SQLite	Observation, DeviceState	ProcessSnapshot, ProcessState	Framebuffer, paleta
Determinismo	Total (fixtures sintéticas)	Total (semilla demo)	Total (snapshots + reloj)	Total (semilla → frame)
Complejidad	Lineal en observaciones	Lineal en observaciones	Lineal en procesos por tick	O(píxeles) / O(1) buscar frame
Pruebas	62 (100%)	138 (100%)	208 (100%)	53/58
Fidelidad	Nivel 4	Nivel 4	Nivel 4	Nivel 4

Observación: No se asignan calificaciones numéricas arbitrarias; la comparación se basa en dimensiones técnicas medibles y verificables.

9. Limitaciones y desarrollo futuro

- **Implementado:** 4 modelos deterministas con pruebas, aplicación web con 6 secciones, informe PDF bilingüe, cartera bilingüe.
- **Incompleto:** El generative engine tiene 5 pruebas fallantes en el proyecto original (no reproducidas en el modelo).
- **Simulado:** Todo el hardware de radio (BLE, Wi-Fi) y /proc se sustituyen por entradas sintéticas en las demostraciones.
- **No disponible sin hardware/servicios:** Escaneo BLE real vía bleak/BlueZ, escaneo Wi-Fi real vía iw/nmcli, lectura real de /proc, almacenamiento SQLite persistente.
- **No verificado:** Ejecución en hardware real de los proyectos originales (requiere adaptadores específicos y permisos).
- **Mejoras futuras:** Canal de integración continua, más backends de escaneo, exportación de GIF/PNG para GenArt, pruebas de propiedades.

10. Conclusiones

El trabajo demuestra capacidad de ingeniería de software en: análisis de código fuente, extracción algorítmica, reconstrucción de modelos deterministas, implementación de pruebas automatizadas, desarrollo de aplicación web interactiva con visualizaciones canvas, generación de informes técnicos PDF y cartera bilingüe. Todas las afirmaciones están respaldadas por código fuente real y pruebas ejecutables. No se fabrican capacidades ni resultados.

Apéndice A — Glosario

- **RSSI:** Received Signal Strength Indicator — intensidad de señal recibida en dBm. Es una medida de fuerza, no de distancia.
- **EMA:** Exponential Moving Average — promedio móvil exponencial para suavizado de señales ruidosas.
- **Jiffies:** Unidades de tiempo del núcleo Linux (CLK_TCK = 100 por defecto en x86).
- **PID reuse:** Reuso de identificadores de proceso cuando un PID se libera y se asigna a otro proceso.
- **AKM:** Authentication Key Management — suites de autenticación en seguridad Wi-Fi (PSK, SAE, OWE).
- **Half-block encoding:** Codificación de terminal donde cada celda representa 2 píxeles verticales (carácter ▀ o █).
- **Splitmix64:** Función de mezcla de 64 bits para generación de números aleatorios deterministas sin estado.

Apéndice B — Ecuaciones

Véase la sección 4 para todas las ecuaciones formales utilizadas en las implementaciones.

Apéndice C — Referencias de código fuente

- Wi-Fi Time Machine: `wtm/scan.py`, `wtm/util.py`, `wtm/viz.py`, `wtm/db.py`, `wtm/export.py`
- BLE Radar: `ble_radar/scan.py`, `ble_radar/lifecycle.py`, `ble_radar/engine.py`, `ble_radar/radar.py`, `ble_radar/util.py`, `ble_radar/replay.py`, `ble_radar/db.py`, `ble_radar/export.py`
- Process Ecosystem Monitor: `process_ecosystem/collector.py`, `process_ecosystem/cpu.py`, `process_ecosystem/model.py`, `process_ecosystem/engine.py`, `process_ecosystem/demo.py`, `process_ecosystem/history.py`
- Terminal Generative Engine: `genart/rng.py`, `genart/color.py`, `genart/framebuffer.py`, `genart/engine.py`, `genart/modes/flowfield.py`, `genart/modes/plasma.py`, `genart/modes/attractor.py`

Apéndice D — Índice completo de proyectos

Véase el capítulo 2 para el inventario completo con estados, tecnologías y estados de implementación de todos los proyectos descubiertos en los repositorios de GitHub y directorios locales.